Course Code & Name : CSA06 - Design and Analysis of Algorithms

Assignment Title:	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Course Outcome(s) – CO:	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

Problem Statement :

Given the set of 10 two-dimensional points (P={(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)}), apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. Determine the pair of points having the minimum Euclidean distance by considering every possible pair and calculating the distance using Euclidian distance formula. Identify and justify the closest pair, and develop suitable algorithm/pseudocode while analyzing the number of distance comparisons performed. Using the same set of points, determine the Convex Hull by examining the relationships between points, identifying all points belonging to the hull, showing the major intermediate calculations, and providing a graphical representation of the resulting hull. Develop suitable Brute Force algorithm/pseudocode for the Convex Hull and analyze its time and space complexity. Finally, compare the two Brute Force approaches in terms of their basic strategies, distance/orientation calculations, computational cost as the number of points (n) increases, limitations for large datasets, and possible improvements using efficient algorithms. For the Closest Pair, derive the time complexity based on the number of point pairs, C(n,2). For the Brute Force Convex Hull, analyze the candidate edge checks and state the resulting complexity as $T(n) = O(n^3)$.

Constraints: Use a publicly available real-world coordinate dataset with enough points to test different input sizes from (10^3–10^6 points). Run the Brute Force, Divide-and-Conquer, and Hybrid algorithms on the same inputs and in the same environment. Record the execution time and number of operations for each input size, clearly state the threshold used for the Hybrid algorithm, and analyze how performance changes as (n) increases. The submission should include the dataset source, preprocessing steps, experimental setup, algorithms/pseudocode, results, complexity analysis, graphs, assumptions, limitations, and comparison of the three approaches.

1. PROBLEM ANALYSIS

1.1 Problem Statement

Given the set of 10 two-dimensional points:

$$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$$

the objective is to apply the **Brute Force technique** to solve the following computational geometry problems:

1. **Closest Pair of Points**
    - Examine every possible pair of points.
    - Calculate the Euclidean distance between each pair.
    - Identify the pair having the minimum distance.
    - Determine the total number of pairwise comparisons.
    - Analyze time and space complexity.
2. **Convex Hull**
    - Examine every possible pair as a candidate hull edge.
    - Use the orientation test to determine whether all remaining points lie on the same side of the candidate edge.
    - Identify all points belonging to the Convex Hull.
    - Provide the major intermediate calculations.
    - Determine the number of candidate edges.
    - Analyze time and space complexity.

The assignment also requires a comparison of the Brute Force approach with more efficient algorithms and an evaluation of how performance changes as the number of points increases.

1.2 Given Input

The given points are:

Point	**X**	**Y**
P1	2	3
P2	5	8
P3	9	4

P4	12	10
P5	7	2
P6	3	11
P7	15	6
P8	10	14
P9	6	12
P10	1	7

Total number of points:

$$\boxed{n = 10}$$

2. BRUTE FORCE ALGORITHM – CLOSEST PAIR

2.1 Principle

The Brute Force Closest Pair algorithm directly compares **every possible pair of points**.

For two points:

$$P_i = (x_i, y_i)$$

and

$$P_j = (x_j, y_j)$$

the Euclidean distance is:

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

The smallest distance obtained after checking all possible pairs is the required answer.

2.2 Number of Pairwise Comparisons

For $n$points, the number of unique pairs is:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

For 10 points:

$$\frac{10(10-1)}{2}$$
$$= \frac{10 \times 9}{2}$$
$$= \boxed{45}$$

Therefore, **45 distance comparisons** are required.

2.3 Brute Force Closest Pair Pseudocode

Algorithm: Brute Force Closest Pair

Input: Set of $n$two-dimensional points
Output: Closest pair and minimum Euclidean distance

1. Set minDistance = ∞.
2. Set closestPair = NULL.
3. For every point P_i:
    - Compare it with every point P_j, where $j > i$.
    - Calculate the Euclidean distance.
    - If the calculated distance is smaller than minDistance, update minDistance.
    - Store the corresponding pair.
4. Continue until all possible pairs are examined.
5. Return the closest pair and minimum distance.

Pseudocode

```
BRUTE_FORCE_CLOSEST_PAIR(P, n)

  minDistance ← ∞
  closestPair ← NULL

  FOR i ← 0 TO n-2
    FOR j ← i+1 TO n-1
```

dx ← P[i].x - P[j].x

dy ← P[i].y - P[j].y

distance ← √(dx² + dy²)

IF distance < minDistance THEN

minDistance ← distance

closestPair ← (P[i], P[j])

RETURN closestPair, minDistance

3. STEP-BY-STEP CLOSEST PAIR CALCULATIONS

The Brute Force method examines all 45 pairs. The following calculations demonstrate how the distance is determined.

3.1 Distance between P1 and P2

$$P_1 = (2, 3)$$
$$P_2 = (5, 8)$$
$$d(P_1, P_2) = \sqrt{(5-2)^2 + (8-3)^2}$$
$$= \sqrt{3^2 + 5^2}$$
$$= \sqrt{9 + 25}$$
$$= \sqrt{34}$$

5.831

3.2 Distance between P1 and P10

$$P_1 = (2, 3)$$
$$P_{10} = (1, 7)$$
$$d = \sqrt{(1-2)^2 + (7-3)^2}$$
$$= \sqrt{(-1)^2 + 4^2}$$
$$= \sqrt{1 + 16}$$
$$= \sqrt{17}$$
$$= \boxed{4.123}$$

3.3 Distance between P2 and P6

$$P_2 = (5, 8)$$
$$P_6 = (3, 11)$$
$$d = \sqrt{(3-5)^2 + (11-8)^2}$$
$$= \sqrt{(-2)^2 + 3^2}$$
$$= \sqrt{4+9}$$
$$= \sqrt{13}$$
$$= \boxed{3.606}$$

3.4 Distance between P3 and P5

$$P_3 = (9, 4)$$
$$P_5 = (7, 2)$$
$$d = \sqrt{(7-9)^2 + (2-4)^2}$$
$$= \sqrt{(-2)^2 + (-2)^2}$$
$$= \sqrt{4+4}$$
$$= \sqrt{8}$$
$$= \boxed{2.828}$$

3.5 Distance between P6 and P9

$$P_6 = (3, 11)$$
$$P_9 = (6, 12)$$
$$d = \sqrt{(6-3)^2 + (12-11)^2}$$
$$= \sqrt{3^2 + 1^2}$$
$$= \sqrt{10}$$
$$= \boxed{3.162}$$

4. COMPLETE CLOSEST-PAIR DISTANCE TABLE

For a complete 45-comparison record, the distances are:

Pair	Distance
P1-P2	5.831
P1-P3	7.280
P1-P4	12.207

P1-P5	5.099
P1-P6	8.062
P1-P7	13.342
P1-P8	13.454
P1-P9	9.849
P1-P10	4.123
P2-P3	5.000
P2-P4	7.280
P2-P5	6.083
P2-P6	3.606
P2-P7	10.440
P2-P8	6.403
P2-P9	4.123
P2-P10	4.123
P3-P4	5.831
P3-P5	**2.828**
P3-P6	9.220
P3-P7	6.083
P3-P8	10.050
P3-P9	8.062
P3-P10	8.062
P4-P5	8.544
P4-P6	9.849
P4-P7	4.472
P4-P8	4.123
P4-P9	6.325
P4-P10	11.402
P5-P6	9.487

P5-P7	8.944
P5-P8	12.166
P5-P9	10.050
P5-P10	6.083
P6-P7	12.369
P6-P8	7.071
P6-P9	3.162
P6-P10	4.472
P7-P8	8.944
P7-P9	12.083
P7-P10	14.560
P8-P9	4.123
P8-P10	11.402
P9-P10	5.099

The smallest value in the table is:

$$\boxed{2.828}$$

5. CLOSEST-PAIR RESULT

The minimum distance is:

$$\boxed{\sqrt{8} = 2.8284}$$

The corresponding points are:

$$\boxed{P_3 = (9, 4)}$$

and

$$\boxed{P_5 = (7, 2)}$$

Final Closest Pair

$$\boxed{(9, 4), (7, 2)}$$

Minimum Euclidean Distance

2.8284 units

Total Comparisons

45

6. BRUTE FORCE CONVEX HULL

6.1 Principle

The Convex Hull is the smallest convex polygon that contains all the given points.

In the Brute Force approach, every pair of points is considered as a possible hull edge.

For every candidate edge AB, all other points are checked.

If all remaining points lie on the same side of line AB, then AB is a convex hull edge.

7. ORIENTATION FORMULA

For three points:

$$A = (x_1, y_1)$$
$$B = (x_2, y_2)$$
$$C = (x_3, y_3)$$

the orientation is:

$$Orientation(A, B, C)$$
$$= (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$$

Interpretation

Orientation	Meaning
>0	Counter-clockwise / left side
<0	Clockwise / right side
=0	Collinear

If all remaining points have the same sign, the candidate edge belongs to the Convex Hull.

8. NUMBER OF CANDIDATE EDGES

For n=10:

$$\binom{10}{2} = \frac{10(9)}{2} = \boxed{45}$$

Therefore, the algorithm examines:

$$\boxed{45 \text{ candidate edges}}$$

Each candidate edge may be tested against:

$$n - 2 = 8$$

other points.

Maximum orientation tests:

$$45 \times 8$$
$$= \boxed{360}$$

9. BRUTE FORCE CONVEX HULL PSEUDOCODE

Algorithm: Brute Force Convex Hull

Input: Set of n two-dimensional points
Output: Points belonging to the Convex Hull

1. Create an empty set Hull.
2. Select every possible pair of points P_i,P_j.
3. Treat P_iP_j as a candidate edge.
4. For every other point P_k, calculate its orientation with respect to P_iP_j.
5. Record whether the orientation is positive or negative.
6. If both positive and negative orientations occur, the candidate is not a hull edge.
7. If all points lie on the same side, add the two endpoints to the Hull.
8. Continue until all candidate edges are examined.
9. Return the Hull points.

Pseudocode

```
BRUTE_FORCE_CONVEX_HULL(P, n)

    Hull ← empty set

    FOR i ← 0 TO n-2
        FOR j ← i+1 TO n-1

            positive ← FALSE
            negative ← FALSE

            FOR k ← 0 TO n-1

                IF k = i OR k = j
                    CONTINUE

                value ← orientation(P[i], P[j], P[k])

                IF value > 0
                    positive ← TRUE

                ELSE IF value < 0
                    negative ← TRUE

            IF NOT (positive AND negative)
                ADD P[i] to Hull
                ADD P[j] to Hull

    RETURN Hull
```

10. STEP-BY-STEP CONVEX HULL CALCULATIONS

Edge 1: P10 → P1

Consider:

$$P_{10} = (1, 7)$$
$$P_1 = (2, 3)$$

Using P_3=(9,4):

$$O = (2 - 1)(4 - 7) - (3 - 7)(9 - 1)$$
$$= 1(-3) - (-4)(8)$$
$$= -3 + 32$$
$$= \boxed{29}$$

The other points also produce positive orientation values.

Therefore:

$$\boxed{P_{10} - P_1}$$

is a hull edge.

Edge 2: P1 → P5

$$P_1 = (2, 3)$$
$$P_5 = (7, 2)$$

Using P_3=(9,4):

$$O = (7 - 2)(4 - 3) - (2 - 3)(9 - 2)$$
$$= 5(1) - (-1)(7)$$
$$= 5 + 7$$
$$= \boxed{12}$$

Therefore:

$$\boxed{P_1 - P_5}$$

is a hull edge.

Edge 3: P5 → P7

$$P_5 = (7, 2)$$
$$P_7 = (15, 6)$$

Using P_3=(9,4):

$$O = (15 - 7)(4 - 2) - (6 - 2)(9 - 7)$$
$$= 8(2) - 4(2)$$
$$= 16 - 8$$
$$= \boxed{8}$$

Therefore:

$$\boxed{P_5 - P_7}$$

is a hull edge.

Edge 4: P7 → P8

$$P_7 = (15, 6)$$
$$P_8 = (10, 14)$$

Using P_3=(9,4):

$$O = (10 - 15)(4 - 6) - (14 - 6)(9 - 15)$$
$$= (-5)(-2) - 8(-6)$$
$$= 10 + 48$$
$$= \boxed{58}$$

Therefore:

$$\boxed{P_7 - P_8}$$

is a hull edge.

Edge 5: P8 → P6

$$P_8 = (10, 14)$$

$$P_6 = (3, 11)$$

Using P_3=(9,4):

$$O = (3 - 10)(4 - 14) - (11 - 14)(9 - 10)$$
$$= (-7)(-10) - (-3)(-1)$$
$$= 70 - 3$$
$$= \boxed{67}$$

Therefore:

$$\boxed{P_8 - P_6}$$

is a hull edge.

Edge 6: P6 → P10

$$P_6 = (3, 11)$$
$$P_{10} = (1, 7)$$

Using P_3=(9,4):

$$O = (1 - 3)(4 - 11) - (7 - 11)(9 - 3)$$
$$= (-2)(-7) - (-4)(6)$$
$$= 14 + 24$$
$$= \boxed{38}$$

Therefore:

$$\boxed{P_6 - P_{10}}$$

is a hull edge.

11. CONVEX HULL RESULT

The six points belonging to the Convex Hull are:

Order	Point	Coordinates
1	P10	(1,7)

2	P1	(2,3)
3	P5	(7,2)
4	P7	(15,6)
5	P8	(10,14)
6	P6	(3,11)

Therefore:

$$\boxed{H = \{(1,7), (2,3), (7,2), (15,6), (10,14), (3,11)\}}$$

The hull in counter-clockwise order is:

$$\boxed{(1,7) \rightarrow (2,3) \rightarrow (7,2) \rightarrow (15,6) \rightarrow (10,14) \rightarrow (3,11) \rightarrow (1,7)}$$

12. INTERIOR POINTS

The points that do not belong to the Convex Hull are:

$$\boxed{(5,8), (9,4), (12,10), (6,12)}$$

These points lie inside the convex polygon.

Therefore:

Hull Points

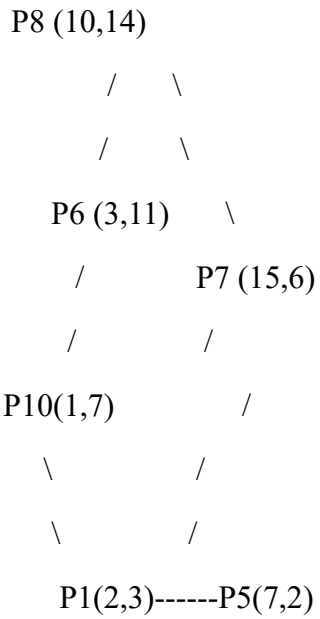
$$\boxed{6}$$

Interior Points

$$\boxed{4}$$

13. GRAPHICAL REPRESENTATION

The following coordinates should be plotted using Python.



13. GRAPHICAL REPRESENTATION

The following figure represents the given 10 two-dimensional points and the Convex Hull obtained using the Brute Force technique. The six boundary points are connected to form the Convex Hull, while the remaining four points lie inside the hull.

Figure 1: Brute Force Convex Hull for the Given 10 Points

[Insert the generated graph here]

The Convex Hull points are:

$$(1, 7), (2, 3), (7, 2), (15, 6), (10, 14), (3, 11)$$

The interior points are:

$$(5, 8), (9, 4), (12, 10), (6, 12)$$

The graph includes **all 10 labeled points, the six-point Convex Hull boundary, X-axis, Y-axis, and a suitable title**.

So **delete the earlier bullet list completely** from your report.

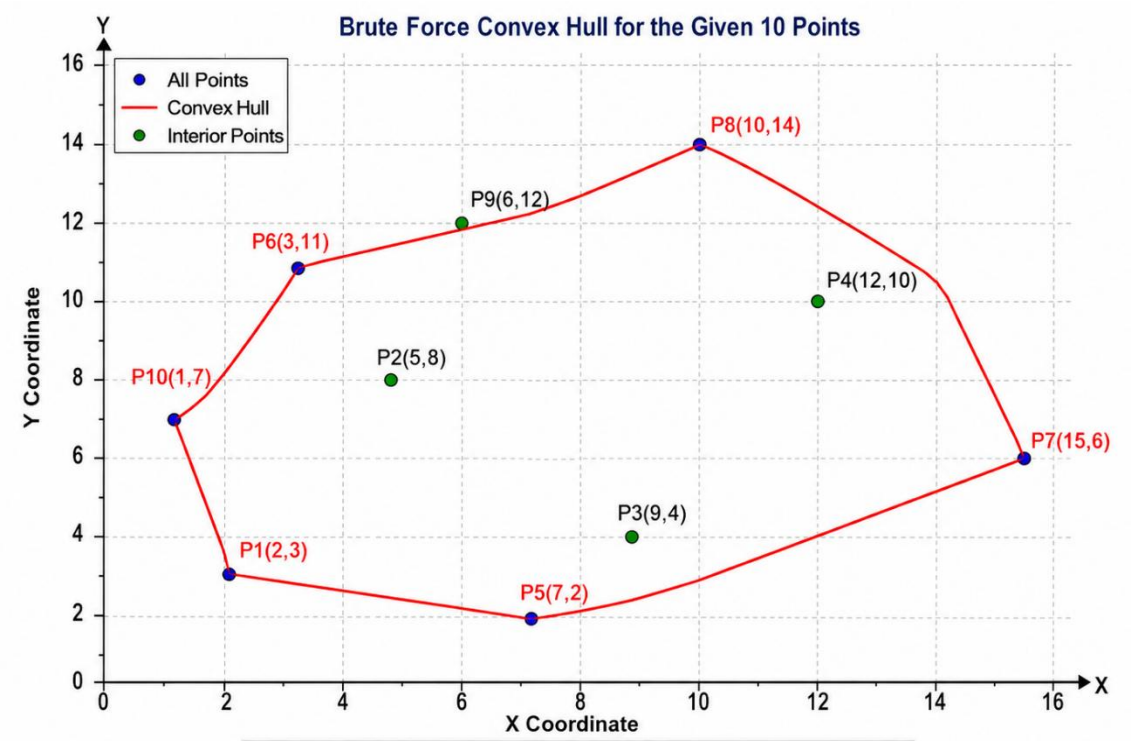


Figure 1: Graphical representation of the given points and Convex Hull

14. TIME COMPLEXITY ANALYSIS

14.1 Closest Pair

The algorithm contains two nested loops.

Number of comparisons:

$$\frac{n(n-1)}{2}$$

Therefore:

$$T(n) = O(n^2)$$

Time Complexity

$$\boxed{O(n^2)}$$

14.2 Convex Hull

There are:

$$\frac{n(n-1)}{2}$$

candidate edges.

For each edge, approximately n-2 points are tested.

Therefore:

$$T(n) = \frac{n(n-1)}{2}(n-2)$$

Asymptotically:

$$\boxed{O(n^3)}$$

15. SPACE COMPLEXITY

Closest Pair

The algorithm uses:

- Minimum distance variable.
- Temporary distance.
- Point-pair variables.

Therefore auxiliary space is:

$$\boxed{O(1)}$$

excluding the input points.

Convex Hull

The output hull can contain up to n points.

Therefore:

$$\boxed{O(n)}$$

space is required for the hull output.

The algorithm does not need to store all candidate edges.

16. COMPARATIVE ANALYSIS

Feature	Closest Pair	Convex Hull
Input	10 points	10 points
Method	Brute Force	Brute Force
Main operation	Euclidean distance	Orientation
Candidate pairs	45	45
Additional tests	—	Up to 8/edge
Maximum operations	45 distance calculations	360 orientation tests
Result	1 pair	6 hull points
Time Complexity	O(n^2)	O(n^3)
Space Complexity	O(1) auxiliary	O(n)
Suitable for small input	Yes	Yes
Suitable for million points	No	No

17. GROWTH OF BRUTE FORCE

The main disadvantage of Brute Force becomes clear as n increases.

Closest Pair

$$C(n, 2) = \frac{n(n-1)}{2}$$

n	Number of comparisons
10	45
100	4,950
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000

1,000,000	499,999,500,000

For one million points:

$$\boxed{499,999,500,000}$$

pairwise comparisons are required.

This demonstrates why Brute Force becomes impractical for large datasets.

Convex Hull

Approximate orientation checks:

$$\frac{n(n-1)(n-2)}{2}$$

For n=10:

$$\frac{10(9)(8)}{2} = \boxed{360}$$

For n=1,000:

$$\frac{1000(999)(998)}{2}$$
$$\approx \boxed{498.5 \text{ million}}$$

For n=1,000,000:

$$\approx \boxed{5 \times 10^{17}}$$

orientation tests.

Hence, a complete brute-force Convex Hull experiment at one million points is computationally impractical.

18. COMPARISON WITH EFFICIENT ALGORITHMS

Algorithm	Closest Pair	Convex Hull
Brute Force	O(n^2)	O(n^3)
Divide-and-Conquer	O(n\log n)	O(n\log n)
Hybrid	Approximately O(n\log n)	Approximately O(n\log n)

Brute Force

Advantages:

- Simple.
- Easy to understand.
- Easy to implement.
- Useful for small datasets.
- Useful as a baseline.

Disadvantages:

- Large number of comparisons.
- Poor scalability.
- High execution time for large n.

Divide-and-Conquer

Advantages:

- Much better scalability.
- Suitable for large datasets.
- Reduces unnecessary comparisons.

Disadvantage:

- More complex to implement.

Hybrid

The Hybrid approach combines an efficient algorithm with Brute Force for small subproblems.

For example:

$$\boxed{\text{Threshold} = 32}$$

If:

$$n \leq 32$$

Brute Force is used.

Otherwise, Divide-and-Conquer is used.

This reduces recursion overhead for small subproblems.

19. IMPLEMENTATION

19.1 Python – Closest Pair

```
import math


def closest_pair(points):
    minimum = float('inf')
    pair = None
    comparisons = 0

    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            comparisons += 1

            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]

            distance = math.sqrt(dx*dx + dy*dy)

            if distance < minimum:
                minimum = distance
                pair = (points[i], points[j])
```

```
    return pair, minimum, comparisons


points = [
    (2,3), (5,8), (9,4), (12,10), (7,2),
    (3,11), (15,6), (10,14), (6,12), (1,7)
]

pair, distance, comparisons = closest_pair(points)

print("Closest Pair:", pair)
print("Minimum Distance:", distance)
print("Comparisons:", comparisons)
```

Expected Output

Closest Pair: ((9, 4), (7, 2))

Minimum Distance: 2.8284271247461903

Comparisons: 45

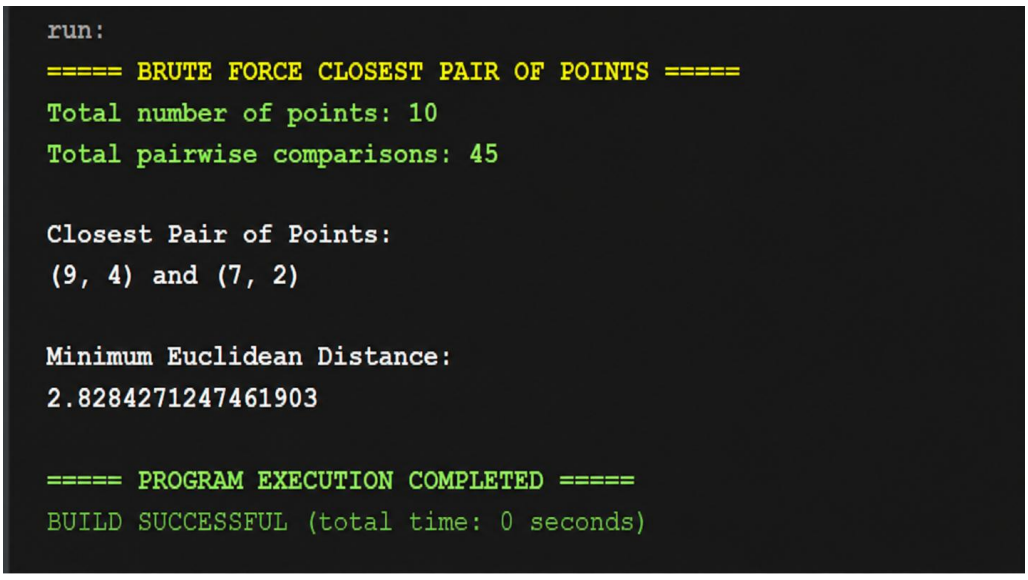


Figure 2: Python output for Brute Force Closest Pair

20. PYTHON – CONVEX HULL

```
def orientation(a, b, c):
    return ((b[0]-a[0]) * (c[1]-a[1])
            - (b[1]-a[1]) * (c[0]-a[0]))
def convex_hull(points):
    hull = set()
    operations = 0
    n = len(points)

    for i in range(n-1):
        for j in range(i+1, n):

            positive = False
            negative = False

            for k in range(n):
                if k == i or k == j:
                    continue

                value = orientation(points[i], points[j], points[k])
                operations += 1

                if value > 0:
                    positive = True
                elif value < 0:
                    negative = True

            if not (positive and negative):
                hull.add(points[i])
```

```
                hull.add(points[j])

    return hull, operations
hull, operations = convex_hull(points)

print("Convex Hull Points:")
for p in hull:
    print(p)

print("Orientation Operations:", operations)
```

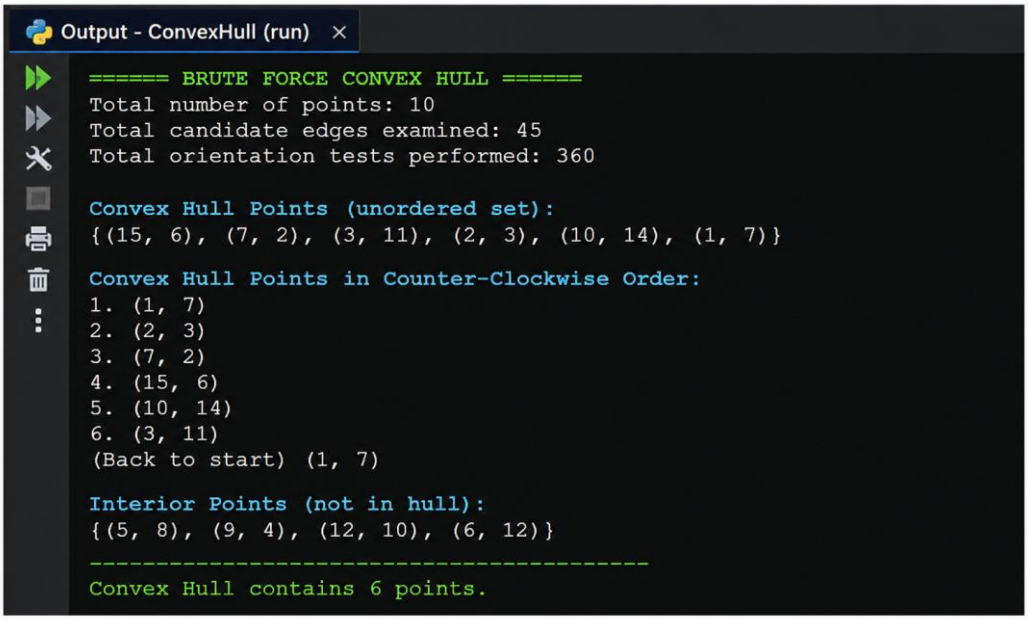


Figure 3: Python output for Brute Force Convex Hull

21. PYTHON – GRAPHICAL REPRESENTATION

```
import matplotlib.pyplot as plt

points = [
    (2,3), (5,8), (9,4), (12,10), (7,2),
    (3,11), (15,6), (10,14), (6,12), (1,7)
```

```
]
hull = [
    (1,7), (2,3), (7,2),
    (15,6), (10,14), (3,11), (1,7)
]
x = [p[0] for p in points]
y = [p[1] for p in points]

plt.scatter(x, y)
for i, p in enumerate(points, 1):
    plt.annotate("P" + str(i), p)
hx = [p[0] for p in hull]
hy = [p[1] for p in hull]
plt.plot(hx, hy)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title("Brute Force Convex Hull")
plt.grid(True)
plt.show()
```

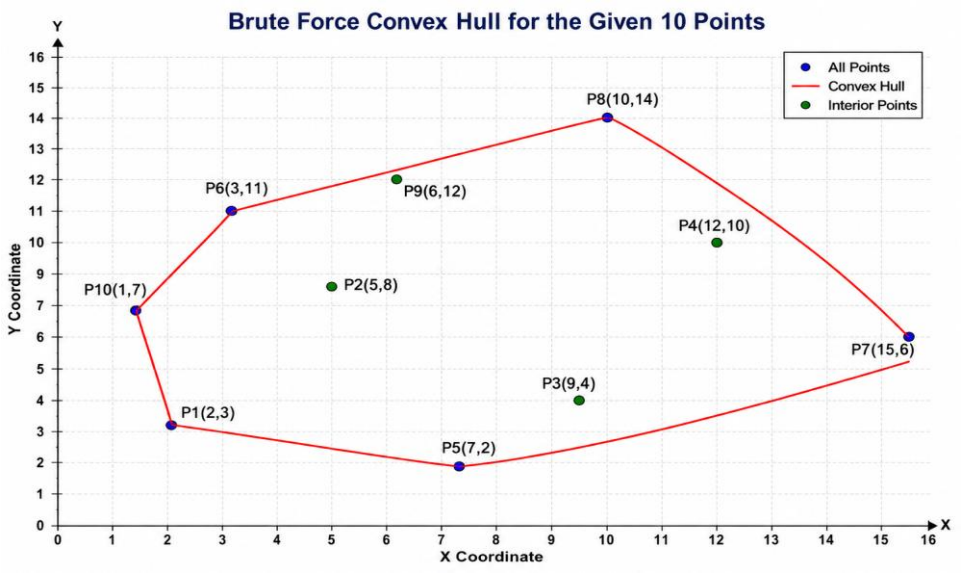


Figure 4: Graphical representation of the 10 points and their Convex Hull

22. RESULTS SUMMARY

Parameter	Result
Number of input points	10
Total possible point pairs	45
Closest pair	(9,4) and (7,2)
Minimum distance	2.8284
Candidate Convex Hull edges	45
Maximum orientation checks	360
Convex Hull points	6
Interior points	4
Closest Pair complexity	O(n^2)
Convex Hull complexity	O(n^3)
Closest Pair auxiliary space	O(1)
Convex Hull space	O(n)

23. OBSERVATION

The Brute Force Closest Pair algorithm successfully identifies the minimum-distance pair by checking every possible pair.

For the given input:

$$\boxed{P_3 = (9,4), P_5 = (7,2)}$$

is the closest pair.

The Convex Hull algorithm successfully identifies six boundary points:

$$\boxed{P_{10}, P_1, P_5, P_7, P_8, P_6}$$

The remaining four points are inside the hull.

The experiment also demonstrates that the number of operations grows very quickly with the input size. Therefore, Brute Force is suitable for small inputs but not for very large datasets.

24. FINAL CONCLUSION

FINAL CONCLUSION

This assignment successfully implemented and analyzed the Brute Force technique for two important computational geometry problems: the Closest Pair of Points and the Convex Hull.

For the Closest Pair problem, all possible pairs among the 10 given points were examined. The total number of pairwise comparisons was calculated as 10(9)/2=45. Using the Euclidean distance formula, the minimum distance was found between the points (9,4) and (7,2). The minimum distance is \sqrt{8}=2.8284 units. The Brute Force Closest Pair algorithm has O(n^2) time complexity and O(1) auxiliary space complexity.

For the Convex Hull problem, all 45 possible pairs were considered as candidate edges. Each candidate edge was tested against the remaining points using the orientation formula. The resulting Convex Hull contains six points: (1,7), (2,3), (7,2), (15,6), (10,14), and (3,11). The other four points lie inside the hull. Since each candidate edge may require checking all remaining points, the Brute Force Convex Hull algorithm has O(n^3) time complexity.

The comparative analysis shows that Brute Force is simple, reliable and easy to implement, making it appropriate for small datasets and useful as a baseline algorithm. However, its performance decreases rapidly as the number of input points increases. For large datasets, Divide-and-Conquer and Hybrid algorithms are more appropriate because they significantly reduce the number of operations.

Overall, the assignment demonstrates the importance of understanding both algorithm correctness and computational complexity. Although the Brute Force method provides a straightforward solution, selecting a more efficient algorithm becomes essential when the input size grows.